FUNCTIONAL PEARL

Rank polymorphism at work

ARTJOMS ŠINKAROVS
University of Southampton, UK
(email: A.Sinkarovs@soton.ac.uk*)*

SVEN-BODO SCHOLZ
Radboud University, Netherlands
(email: SvenBodo.Scholz@ru.nl*)*

Abstract

This pearl demonstrates how rank polymorphism enables programmers to abstract performance-tuning aspects into the ranks and shapes of function arguments. Using prefix sum as an example, we show how a single, generic rank-polymorphic specification can represent an entire class of traditionally separate algorithmic specifications that differ in performance characteristics but not in the overall outcome.

The elegance of this approach lies in the observation that, once encoded within a dependently-typed language, we can prove that certain classes of argument reshaping do not affect the overall computational outcome. Consequently, this establishes the equivalence of all corresponding algorithmic specifications, which in classic (typically imperative) settings would be extremely tedious, if feasible at all.

1 Introduction

Rank polymorphism is a language feature that permits the definition of functions operating on arrays of arbitrary rank, where the rank of an array is the number of its dimensions (axes). As an illustration, consider a function that adds two numeric arrays element-wise; the same definition applies to 0-dimensional, 1-dimensional, . . . arrays. A common justification for supporting rank polymorphism is its facilitation of generic programming: rather than providing a separate definition for each array rank, a single definition suffices for all ranks. This reduces the likelihood of errors, curtails verbosity, and—because compositions of rank-polymorphic operations yield more general specifications—encourages programmers to contemplate corner cases that might otherwise be overlooked (Iverson 1980). Arguably, this is a principal reason why specifications in the APL (Iverson 1962) family are exceptionally terse.

Although this interpretation of rank polymorphism dates back to the 1960s, the present pearl offers a fresh perspective on the core idea, showing that, when rank polymorphism is made to work harder, it yields considerably richer benefits.

First, rank polymorphism gives rise to a notion of algorithmic equivalence. For a given rank-polymorphic function one can often define an equivalence relation on array shapes such that reshaping an array within the same equivalence class does not affect the computed values. A simple example is the sum of all array elements—the resulting value remains unchanged regardless of the shape assigned to the input array (provided that reshapes neither add nor remove elements).



© 2026 Copyright held by the owner/author(s).
This work is licensed under a Creative Commons Attribution 4.0 International License.

Journal of Functional Programming, Volume 36, Article 20 DOI: 10.46298/jfp.17992
Submitted December 2024; accepted October 2026

Second, if array shapes are endowed with a richer structure than a mere sequence of dimensions, rank polymorphism becomes a tool for performance engineering. As Okasaki (1998) demonstrated, the choice of data structures determines algorithmic complexity in functional languages. Analogously, we show that the choice of array shape influences the complexity of array-based algorithms. Consider a rank-polymorphic function together with a shape equivalence relation such that all inputs in the same class produce identical results. Assuming a fixed evaluation strategy and runtime costs, reshapes within the equivalence class dictate the actual execution time while preserving the computed values. In other words, a rank-polymorphic specification defines *what* is computed, whereas the choice of shape within an equivalence class determines *how* the computation proceeds. Consequently, finding an efficient algorithm amounts to searching for a shape that minimises the runtime cost; in this paper we compare candidate shapes by evaluating a cost model. In principle, such cost models can encode assumptions about the underlying hardware, such as parallelism, memory hierarchy or cache sizes; here we only model the number of operations and parallelism. Other aspects, such as the layout of arrays in memory or the sparsity of the data, are determined by the choice of array representation rather than by shapes, and our cost model does not account for them.

The principal technical contribution of this work is a precise formulation of the above ideas via a framework in Agda, which enables us to define: (i) rank-polymorphic arrays (Section 2); (ii) equivalence classes of shapes (Section 4); and (iii) runtime-cost models (Section 6).

Using the scan algorithm as a running example, we (i) encode a rank-polymorphic version of scan; (ii) prove that all scans belonging to a chosen shape-equivalence class compute identical results; (iii) devise a cost model for an idealised multicore processor; and (iv) demonstrate the runtime impact of reshaping.

When we introduce rank-polymorphic arrays, we deliberately avoid fixing the representation of arrays to any particular data structure. We achieve this by specifying an array theory that stipulates assumptions about shapes and indices, whilst requiring the concrete array datatype to be a representable functor. Our running example and accompanying equivalence proof are presented without committing to a specific model, meaning that they can be instantiated for any admissible representation. Thus, we may (a) define array representations and basic combinators in Agda; (b) perform performance engineering to select optimal array shapes for a given representation; and (c) subsequently instantiate our algorithm for that representation, preserving the established correctness guarantees. In the remainder of the paper we guide the reader through the rationale behind our design choices and the technical details of each component of the framework. This paper is an Agda script, available online (Šinkarovs and Scholz 2026); it has been checked with Agda 2.8.0 (Agda Development Team 2026) and the Agda standard library version 2.3 (The Agda Community 2026).

2 Arrays

In array programming the structure of array shapes is of central importance. The richer the shape structure, the more expressive the array combinators that can be derived. Conversely, poorly chosen shape encodings may inhibit expressivity and reasoning. To motivate this, we consider definitions of top-down and bottom-up *shape-directed folds*, which frequently arise in divide-and-conquer algorithms: they recurse over the shape and reduce an array one dimension at a time.

We work in a dependently-typed setting and require array types to guarantee index safety. A natural way to achieve this is to index an array type by its shape. Shapes are usually represented as a sequence of natural numbers, each specifying the bound of the index along a particular dimension. The crucial question, however, is which data structure should be used to represent shapes so that the definitions of our folds are not obscured by encoding artefacts.

At first sight, List appears to be a sensible choice. Consider the following abstract array type:

$$\mathrm{Ar} : \mathrm{List}\ \mathbb{N} \to \mathrm{Set} \to \mathrm{Set}$$

where the first argument denotes the shape and the second argument denotes the element type. We deliberately omit a concrete representation of the array itself.

With this definition, a top-down fold can be expressed as follows:

```
top-down : (∀ {n s} → Ar ([ n ] ++ s) X → Ar s X) → (Ar [] X → X)
         → ∀ s → Ar s X → X
top-down f z []       a = z a
top-down f z (n :: s) a = top-down f z s (f a)
```

if we know how to reduce n s-shaped arrays into a single s-shaped array (the first argument f) and we know how to extract the element from an array of the empty shape[1] (the second argument z), then we can reduce arrays of any shape into X by repeatedly applying[2] f followed by a single application of z.

The bottom-up case, where we wish to reduce an s-shaped array of n-element arrays to an s-shaped array, is considerably less straightforward:

```
bottom-up : (∀ {n s} → Ar (s ++ [ n ]) X → Ar s X) → (Ar [] X → X)
          → ∀ s → Ar s X → X
```

The difficulty stems from the fact that Lists are built by pre-pending elements on the left. Consequently, pattern matching on a list proceeds left-to-right, whereas a bottom-up fold needs to take the shape apart from the right. This problem is well known (Swierstra 2022); one solution proposed in the literature (McBride and McKinna 2004) introduces a view type:

```
data View : List ℕ → Set where
  []    : View []
  _::'_ : (s : List ℕ) → (n : ℕ) → View (s ++ [ n ])

view : (xs : List ℕ) → View xs
```

```
{-# TERMINATING #-}
bottom-up f z s a with view s
... | []      = z a
... | s ::' n = bottom-up f z s (f a)
```

We use view to turn the list into the view type, and then we pattern-match on it. Agda does not recognise that bottom-up is terminating, hence we use explicit annotation. We can fix this by defining a measure on the length of the shape that decreases on every recursive call.

Here we can see that representing shapes as Lists is suboptimal. Firstly, view types together with the termination argument would clutter any reasoning about bottom-up folds, making it considerably more involved than reasoning about top-down ones. Secondly, the shape itself

[1] Arrays of the empty shape are scalars: the empty product of index ranges contains exactly one index, so Ar [] X is isomorphic to X rather than to the unit type. Note that Ar [1] X is isomorphic to X as well, but it has rank one.

[2] In the second equation of top-down, the argument a has shape (n :: s), but f expects the shape [n] ++ s. However, the latter normalises to (n :: s), therefore f can be applied to a.

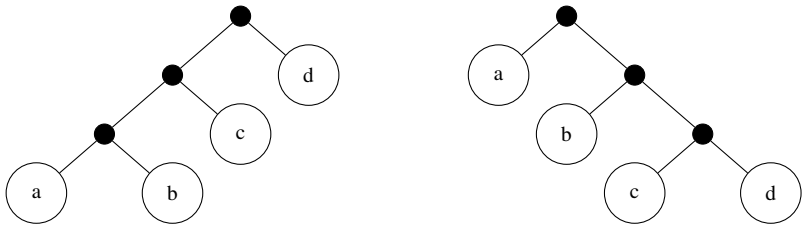
does not convey information about the direction of recursion; consequently we must write two separate definitions, as the direction is dictated by the code rather than by the data.

Independently of the choice of shape structure, there is a further concern: even though we have left the representation of arrays abstract, the above array type is *not* representation-independent, because it commits to a particular encoding of shapes as Lists. Representation independence is crucial, since the interplay between hardware architecture and algorithm determines the preferred array storage. For example, many numerical problems involve arrays in which large portions of the elements are zero, giving rise to compressed storage schemes that omit the zeros and traverse only the non-zero entries. On systems with hierarchical memory, it is important to tile arrays into blocks that are stored contiguously in memory. Hence, we aim to formulate array types and algorithms orthogonal to the data layout, allowing the layout to be chosen later according to the specific requirements.

2.1 Array theory

In this subsection we propose an alternative approach that avoids the shortcomings identified above.

We resolve the issue of recursion direction by encoding shapes as binary trees. In this formulation the shape alone determines whether a fold proceeds top-down or bottom-up. A shape thus describes the logical structure of an array, *i.e.* how its elements are grouped, and this structure guides the recursion. Note that a shape does not prescribe how the array is laid out in memory; this is left to the array representation.



A left-nested tree such as $(((a,b),c),d)$—the left diagram—yields a bottom-up fold, whereas a right-nested tree such as $(a,(b,(c,d)))$—the right diagram—induces a top-down fold. Moreover, this representation can express shapes that cannot be captured by simple lists, for example $((a,b),(c,d))$.

To connect such trees with array elements, we view arrays as containers (Altenkirch et al. 2015) whose shapes are binary trees: every shape determines a set of positions, and an array assigns an element to every position. We achieve representation independence of arrays and shapes by defining a minimalist *array theory*. In this case we fix an interface to shapes, indices and arrays; we define some constraints, but the actual implementation of these structures will be given by concrete models. We define our array theory as follows:

```
record Array : Set₁ where
  field
    S    : Set                    ι-↔   : P (ι n) ↔ Fin n
    P    : S → Set                ⊗-↔   : ∀ {s p} → P (s ⊗ p) ↔ (P s × P p)

    ι    : ℕ → S                  Ar    : S → Set → Set
    _⊗_  : S → S → S              Ar-↔  : ∀ {s} → (P s → X) ↔ Ar s X
```

The theory draws inspiration from container theory (Abbott et al. 2003), introducing restrictions on the kind of shapes that are allowed, and generalising the notion of container interpretation.

The theory consists of shapes S and positions (indices) P within such shapes. We require two constructors for S: ι which lifts a given natural number into S and _⊗_ which builds a shape out of left and right sub-shapes.

Positions P of a given shape s characterise all the valid indices into arrays of that shape. We require two properties of P. Firstly, ι-↔ requires indices of a singleton shape $\iota\ n$ to be isomorphic (denoted by ↔) to the natural numbers $[0 \ldots n)$ which is denoted by the type Fin n. Secondly, ⊗-↔ requires indices of a product shape to be isomorphic to the product of left and right sub-indices.

For readers familiar with universes, our theory says that (S, P) is a monoidal[3] universe where (ι 1) is a unit.

Ar is a type of arrays and Ar-↔ requires Ar of shape s to be isomorphic to a function from indices of shape s to array elements. Note a distinction with the container theory, where the interpretation of the container (S, P) for some X is fixed to $\Sigma\ (s : S)\ (P\ s \to X)$. Here we only say that Ar *has to be* a representable functor (Grothendieck 1960) (*i.e.* (Ar s) is isomorphic to $(P\ s \to X)$). The concrete storage layout is left to the model.

2.2 Array operations

For any array model A, we can expand the isomorphisms of the array theory into operations on shapes, indices, and arrays. We group these operations into a module that we call ArOps which is parametrised by A:

```
module ArOps (A : Array) where
```

```
imap : (P s → X) → Ar s X        ϕ : Fin n → P (ι n)        fst  : P (l ⊗ r) → P l
_[_] : Ar s X → (P s → X)        ψ : P (ι n) → Fin n        snd  : P (l ⊗ r) → P r
                                                            _⊛_ : P l → P r → P (l ⊗ r)
```

The forward direction of the Ar-↔ isomorphism gives us a function that we call imap (sometimes called *tabulate*); the backward direction gives us selections into arrays for which we use a C-like notation _[_]. Similarly, when expanding ι-↔ and ⊗-↔ we obtain convenient operations on indices: ϕ and ψ which translate between the one-dimensional indices and Fin n; fst and snd project left and right sub-indices from the index of a product shape; and _⊛_ makes an index of a product shape from two sub-indices of the corresponding sub-shapes.

2.2.1 Equality of arrays

Using selection, we can define the notion of point-wise equality of two arrays $_\approx_a_$ and show that $\approx_a$ is an equivalence relation, meaning that Ar with $\approx_a$ is a setoid. Throughout the paper we use the term setoid in its type-theoretic sense: a type equipped with an equivalence relation, which need not be proof-irrelevant.

```
_≈ₐ_ : Ar s X → Ar s X → Set
a ≈ₐ b = ∀ ix → a [ ix ] ≡ b [ ix ]
```

```
refl-≈ₐ  : a ≈ₐ a
sym-≈ₐ   : a ≈ₐ b → b ≈ₐ a
trans-≈ₐ : a ≈ₐ b → b ≈ₐ c → a ≈ₐ c
```

[3] A universe $(U,\ El)$ with a monoid structure $(U, \epsilon, *)$ that El maps to the monoid (Set, ⊤, _×_): $El\ \epsilon$ is ⊤ and $El\ (a * b)$ is $El\ a \times El\ b$. Technically, as in monoidal categories, all of these laws hold up to isomorphism.

2.3 Models

The array theory of Section 2.1 imposes only the essential constraints needed for the subsequent reasoning, thereby accommodating a broad spectrum of models. To illustrate this flexibility we sketch several alternatives (omitting the isomorphism proofs). Firstly, we can represent shapes as lists of natural numbers, and we can represent arrays as (shape-indexed) free monads of the Vec type from Agda's standard library:

```
data Tensor (X : Set) : List ℕ → Set where
  pure : X → Tensor X []
  roll : Vec (Tensor X s) n → Tensor X (n :: s)
```

Putting this together we have a model:

$$\mathrm{S} = \mathrm{List}\ \mathbb{N}; \mathrm{P} = \mathrm{All}\ \mathrm{Fin}; \iota = [_]; _\otimes_ = _{+}{+}_; \mathrm{Ar}\ s\ X = \mathrm{Tensor}\ X\ s.$$

This model corresponds to the familiar encoding of multi-dimensional arrays as nested lists (or vectors) in functional languages, such as lists of lists of lists for three-dimensional data.

As another example, let us consider a flat vector model:

$$\mathrm{S} = \mathbb{N}; \mathrm{P} = \mathrm{Fin}; \iota = \mathrm{id}; _\otimes_ = _{*}_; \mathrm{Ar}\ n\ X = \mathrm{Fin}\ n \to X.$$

Shapes are natural numbers (tree structure is not preserved), positions are offsets given by Fin, the shape product is a multiplication of natural numbers, and arrays are stored as functions. This model mirrors the flat, row-major storage used by most array libraries, where a multi-dimensional array is a single contiguous buffer accessed through computed offsets; arrays as functions from offsets also correspond to the delayed (or pull) arrays found in embedded array languages.

A more traditional representation of vectors can be achieved by choosing a model where arrays are stored as cons lists (again using the standard library's Vec):

$$\mathrm{S} = \mathbb{N}; \mathrm{P} = \mathrm{Fin}; \iota = \mathrm{id}; _\otimes_ = _{*}_; \mathrm{Ar}\ n\ X = \mathrm{Vec}\ X\ n$$

This is the representation of choice when data is consumed sequentially, for example when arrays are produced or traversed as streams.

We can also encode less obvious choices as a model of the array theory: in high-performance computing it is often beneficial to use compressed formats, where arrays do not store zeroes, *e.g.* for sparse matrices arising in scientific computing or graph algorithms. In this case, for X with decidable equality, an array can be encoded by three components: a default element that is not stored (zero); a strictly increasing list of the positions (ordered, for example, by their row-major offsets) at which the array holds non-default elements; and a list of the elements stored at these positions. The element at a listed position is the corresponding stored value, and the element at any other position is the default one. Decidable equality of X is needed to construct such an array from a function: we only store the elements that differ from the default.

The multitude of possible models suggests that it would be advantageous to identify a universal model in which we can develop our constructions and subsequently transfer them to all other models. Such a model is presented in the following subsection. Note that only constructions transfer, not proofs, as we discuss in Section 2.3.2.

2.3.1 *Initial model*

Let us define a smallest model that satisfies all the equations of the theory. Following terminology from model theory, we call such a model *initial*. This model has the property that any expression written in it can be lifted into any other model of the same theory. Moreover, this lifting is unique up to isomorphism. We define the initial model for our array theory and we demonstrate its initiality by defining lifting operations.

Shapes are defined as binary trees where ι constructs leaves and _⊗_ combines two sub-trees into a new one. Positions are predicates over shapes also given by two constructors: ι and _⊗_. A position for the shape $\iota\ n$ can be constructed by providing an index of type Fin n. Positions of product shapes can be constructed if positions of the left- and right-sub-shapes can be constructed. Arrays of shapes s with elements of type X are functions from P s into X.

```
module I where

data S : Set where
  ι   : ℕ → S
  _⊗_ : S → S → S

data P : S → Set where
  ι   : Fin n → P (ι n)
  _⊗_ : P s → P p → P (s ⊗ p)

Ar : S → Set → Set
Ar s X = P s → X
```

Now we use the data types and constructors introduced above to define the model that we call A. We omit the definition for ι-↔, ⊗-↔, Ar-↔ as they hold trivially.

```
A : Array
A .Array.S = S
A .Array.P = P
A .Array.ι   = ι
A .Array._⊗_ = _⊗_
A .Array.Ar = Ar
```

Note that we place these definitions in the module named I, obtaining a namespace that we will use to refer to initial shapes, positions, *etc*. in other contexts.

2.3.2 *Initiality*

We show that expressions defined within the initial model can be lifted into any other model that we denote A. As before, we put all the lifting operations into the module called FromInit that is parametrised by A.

Firstly, we define the operation ⟦_⟧s which lifts initial shapes (of type I.S) into the shapes defined in A (of type S).

```
module FromInit (A : Array) where

⟦_⟧s : I.S → S
⟦ I.ι n ⟧s   = ι n
⟦ s I.⊗ p ⟧s = ⟦ s ⟧s ⊗ ⟦ p ⟧s
```

Note that in general we cannot translate S shapes into I.S shapes because S may contain shapes that are not representable in I.S. Also, for most of the models, ⟦_⟧s is not going to be injective, *e.g*. take S to be ℕ and ⊗ to be *. This is why constructions can be transferred from the initial model into A, but proofs cannot: a property of the initial model, such as every shape being built from ι and _⊗_, need not hold in A. Properties of transferred constructions have to be lifted explicitly, as we do in Sections 4 and 5.

Next, ⟦_⟧p translates initial positions (of type I.P s) into positions in the model A (of type P) of lifted shapes (⟦ s ⟧s). As translated positions are of a lifted shape, we can define an

inverse of ⟦_⟧p that we call ⟦_⟧p'.

```
⟦_⟧p : ∀ {s} → I.P s → P ⟦ s ⟧s        ⟦_⟧p' : ∀ {s} → P ⟦ s ⟧s → I.P s
⟦ I.ι i ⟧p    = ϕ i                    ⟦_⟧p' {I.ι n}     i = I.ι (ψ i)
⟦ i I.⊗ j ⟧p = ⟦ i ⟧p ⊛ ⟦ j ⟧p        ⟦_⟧p' {s I.⊗ p} i = ⟦ fst i ⟧p' I.⊗ ⟦ snd i ⟧p'
```

Finally, ⟦_⟧a lifts initial arrays of initial shapes into arrays of lifted shapes. Similarly to position lifting, ⟦_⟧a has an inverse ⟦_⟧a' from the arrays of *lifted* shapes.

```
⟦_⟧a : ∀ {s} → I.Ar s X → Ar ⟦ s ⟧s X        ⟦_⟧a' : ∀ {s} → Ar ⟦ s ⟧s X → I.Ar s X
⟦ a ⟧a = imap λ i → a ⟦ i ⟧p'                ⟦_⟧a' a i = a [ ⟦ i ⟧p ]
```

We show that array and position lifting are inverses by proving the following four equalities[4]:

```
⟦ ⟦ i ⟧p ⟧p' ≡ i        ⟦ ⟦ i ⟧p' ⟧p ≡ i        ⟦ ⟦ a ⟧a ⟧a' I.≈ a        ⟦ ⟦ a ⟧a' ⟧a ≈ a
```

3 Defining scan

We use generalised prefix sums (typically referred to as *scan*) as a running example that helps us demonstrate the proposed technique. Mathematically, prefix sums on vectors of numbers can be specified as

$$s_i = \sum_{j<i} a_j$$

For a given input vector a, we compute a vector s (of the same length as a) whose i-th element is the sum of the i-th prefix of a, *i.e.* all the elements of a up to, but excluding, index i. Such a prefix sum is called *exclusive*; if the element a_i is included in the sum as well, it is called *inclusive*. In the remainder of the paper we concentrate on the exclusive version. No information is lost by this choice: our scan also returns the sum of all the elements, so the inclusive version can be recovered by dropping the first element of the result and appending the total.

This specification can be generalised to vectors containing elements of an arbitrary type X, provided with a binary operation on X that replaces addition and a neutral element of type X that replaces zero. Such a generalisation is called *scan*.

3.1 Scan on vectors

We begin with the definition of scan on one-dimensional arrays, which are often called vectors. We introduce the Vec type as a shorthand for one-dimensional arrays in the initial model. The first argument of Vec is the length of the vector and the second argument is the type of the vector elements; note that this is the opposite order to the standard library's Vec used in Section 2.3. We also define the standard operations on vectors that will be used in the scan: nil[5] constructs the empty vector; cons constructs a vector of length 1 + n given an element and a vector of length n; the two standard destructors, head and tail, are denoted by hd and tl respectively.

```
Vec : ℕ → Set → Set        nil : Vec 0 X        hd : Vec (suc n) X → X
Vec n X = Ar (ι n) X       nil (ι ())           hd v = v (ι zero)
```

[4] Agda struggles to infer hidden variables s in some cases, so in the actual code we have to supply them explicitly, but we omit them here for readability.

[5] The definition uses the absurd pattern (see https://agda.readthedocs.io/en/v2.8.0/language/function-definitions.html#absurd-patterns for more details), because Agda recognises that an index of type Fin 0 could not possibly exist.

```
tl : Vec (suc n) X → Vec n X
tl v (ι i) = v (ι (suc i))

cons : X → Vec n X → Vec (suc n) X
cons x v (ι zero)    = x
cons x v (ι (suc i)) = v (ι i)
```

With these definitions at hand, scan on vectors can be defined as follows:

```
scan : (X → X → X) → X → Vec n X → X × Vec n X
scan {n = 0}       f acc v = acc , nil
scan {n = suc _} f acc v = let s , v' = scan f (f acc $ hd v) (tl v)
                           in s , cons acc v'
```

The function takes the binary operation f as its first argument, a running value *acc* (initially the neutral element of f) as its second argument and the n-element vector v as the third one. The function produces a tuple where the first projection is a reduction of f over v; and the second projection is a vector where each i-th index contains the reduction of f over the prefix of v up to (but not including) the i-th element.

In the case when the length of v is zero ($n = 0$), the second projection is the empty vector. When the input length is a successor of some number, we call scan recursively on the tail of v, adding the head of v to the *acc*umulator. We update this result by prepending the old value of the *acc*umulator to the second projection of the returned pair.

Note that the primary purpose of the scan presented here is to provide a specification of a one-dimensional scan algorithm. This version will serve as a reference point in Sections 3.2 and 5 when proving the correctness of other scan implementations. Consequently, representing vectors using initial arrays (implemented as functions) is appropriate: it simplifies later proofs, and we do not assume that this particular code will be executed at runtime.

Finally, observe that the binary operation f used in scan has arguments of the same type, which differs from the conventional definitions of scan found in the standard libraries of functional languages such as Agda or Haskell. Throughout the remainder of the paper we restrict ourselves to scans on associative operations, for which the arguments must indeed share a common type. Nevertheless, it is worth mentioning that the usual, more general form of scan can be recovered[6] through scanning on function composition.

3.2 Array combinators

We have seen the definition of `scan` on vectors (initial one-dimensional arrays), and now we wish to derive a rank-polymorphic version for all models.

We express a rank-polymorphic version of `scan` using a suite of primitive array combinators such as `map`, `zip`, *etc*. In principle, all of these combinators can be defined in the initial model and then lifted into the model of interest. However, doing so may introduce considerable inefficiency. As a practical illustration, consider an array representation that does not store zeroes. In that case, point-wise summation of two arrays needs only inspect the non-zero

[6] A general version of scan uses a binary operation of type $X \to Y \to Y$. We recover this version by scanning with a (non-dependent) function composition as the binary operation. We compute a composition of partial applications of f for each index, and then apply the resulting functions to the neutral element:

```
scan' : (X → Y → Y) → Y → Vec n X → Y × Vec n Y
scan' f e a = let s , a' = scan _∘'_ id (f ∘' a) in
              s e , (λ i → a' i e)
```

elements of both arrays. By contrast, a naïve point-wise summation would traverse the entire index space, thereby accessing every element of both argument arrays.

To retain representation independence, we abstract over concrete implementations of the combinators. We define a record called Comb that specifies the interfaces to array combinators; this record is parameterised by the array model A. This is analogous to defining a type class in Haskell, meaning that *different* implementations of the combinators can be chosen for each array model. The definition of Comb follows:

```
record Comb : Set₁ where
  field
    K      : X → Ar s X
    map    : (X → Y) → Ar s X → Ar s Y
    nest   : Ar (s ⊗ p) X → Ar s (Ar p X)
    unnest : Ar s (Ar p X) → Ar (s ⊗ p) X
    unzip  : Ar s (X × Y) → Ar s X × Ar s Y
    zipWith : (X → Y → Z) → Ar s X → Ar s Y → Ar s Z
    scanv  : (X → X → X) → X → Ar (ι n) X → X × Ar (ι n) X
```

K returns a constant array whose elements are all equal to its argument; map applies a supplied function to every element of an array; nest and unnest convert between arrays of product shapes and nested arrays; unzip and zipWith behave like their list-based counterparts; scanv implements `scan` on rank-one arrays. Note that Ar of a fixed shape constitutes an applicative functor (McBride and Paterson 2008), which gives rise to K, map, zipWith and unzip. The nest/unnest combinators stem from the monoidal structure of shapes, and scanv is specific to our use-case.

At present the array combinators impose no behavioural constraints. Consequently, different models could choose implementations of the same combinator that yield divergent results, making it impossible to reason about the values produced by a generic (model-independent) array expression that employs these combinators. We resolve this issue by providing canonical implementations for the combinators in any model. These implementations are collected in the record CanonComb, defined as follows:[7]

```
CanonComb : Comb A
CanonComb .K x          = imap λ i → x
CanonComb .map f a      = imap (f ∘ a [_])
CanonComb .nest a       = imap λ i → imap λ j → a [ i ⊛ j ]
CanonComb .unnest a     = imap λ i → a [ fst i ] [ snd i ]
CanonComb .unzip f      = CanonComb .map proj₁ f , CanonComb .map proj₂ f
CanonComb .zipWith f a b = imap λ i → f (a [ i ]) (b [ i ])
CanonComb .scanv f e a  = let s , a' = Vec.scan f e ⟦ a ⟧a' in s , ⟦ a' ⟧a
```

[7] We use copatterns (see https://agda.readthedocs.io/en/v2.8.0/language/copatterns.html) to define the fields of the CanonComb record.

Note that scanv is defined using the implementation given in Section 3.1, by lifting the implementation from the initial model. All canonical combinators compute a function from indices to values, which is then turned into an actual array by imap.

CanonComb supplies default implementations of the combinators for every model, yielding two benefits. First, any array expression that uses combinators can be instantiated in any model by employing these defaults. Second, and more importantly, CanonComb endows the combinator instances with a well-defined semantics. If a model provides more efficient implementations for certain combinators, these may be used provided they preserve the semantics. To formalise this requirement we introduce the record CombOk, which is parameterised by the array model A and by a concrete implementation of the combinators for A, denoted Native. We demand that every native combinator compute the same values (as defined by $_\approx_a_$) as the canonical implementation.

```
record CombOk : Set₁ where
  field
    K-ok       : (x : X) → (Native.K x : Ar s X) ≈ₐ Canon.K x
    map-ok     : (f : X → Y) (a : Ar s X) → Native.map f a ≈ₐ Canon.map f a
    nest-ok    : (a : Ar (s ⊗ p) X) → Native.nest a ≈ₐ Canon.nest a
    unnest-ok  : (a : Ar s (Ar p X)) → Native.unnest a ≈ₐ Canon.unnest a
    unzip-ok   : (a : Ar s (X × Y))
               → Pointwise _≈ₐ_ _≈ₐ_ (Native.unzip a) (Canon.unzip a)
    zipWith-ok : (f : X → Y → Z) (a : Ar s X) (b : Ar s Y)
               → Native.zipWith f a b ≈ₐ Canon.zipWith f a b
    scanv-ok   : (f : X → X → X) (e : X) (a : Ar (ι n) X)
               → Pointwise _≡_ _≈ₐ_ (Native.scanv f e a) (Canon.scanv f e a)
```

We prefix native combinators that originate from a parameter with Native and canonical combinators with Canon.[8] The comparison of products in unzip-ok and scanv-ok uses Pointwise, which takes two relations (*e.g.* R_1 and R_2) and relates the first components of a pair via R_1 and the second components via R_2.

Obviously, CanonComb satisfies CombOk trivially; this is witnessed by CanonCombOk of the following type (the definition is omitted for brevity):

```
CanonCombOk : CombOk A (CanonComb A)
```

3.3 *Generalised scan*

We define `scan` for a given array model A, a collection of combinators N, and a proof that the combinators respect the intended semantics (witnessed by Ok) as follows. The definition of `scan` itself does not use Ok; we nevertheless make it a parameter so that `scan` cannot be instantiated with combinators that are not known to behave correctly. This proof is what allows us to reason about `scan` in arbitrary models in Section 3.5.

```
module Scan (A : Array) (N : Comb A) (Ok : CombOk A N) where
```

[8] Canon is merely a wrapper for (CanonComb A).

```
scan : (s : I.S) → (X → X → X) → X → Ar ⟦ s ⟧s X → X × Ar ⟦ s ⟧s X
scan (I.ι n)     f e a = scanv f e a
scan (s I.⊗ p) f e a = let
                         m , a' = unzip (map (scan p f e) (nest a))
                         r , m' = scan s f e m
                         a''    = zipWith (map ∘ f) m' a'
                       in r , unnest a''
```

Note that scan is defined for arrays of the *lifted* shape s (of type I.S). This is necessary because scan is defined inductively on shapes: for singleton shapes we apply scanv; for product shapes we recurse into sub-arrays, scan the array of their *carries* (the reductions of the sub-arrays, *i.e.* the first components of their scans), and then update the sub-arrays with the scanned carries. This induction works only if we can decide whether a shape is a product or a singleton. In some models this decision is unavailable (*e.g.* when shapes are natural numbers), hence we employ lifted shapes.

We also provide two projections, scan_1 and scan_2, for the carry (the reduction of the whole array) and for the scanned array respectively:

$$\text{scan}_1\ s\ f\ e\ a = \text{scan}\ s\ f\ e\ a\ .\text{proj}_1 \qquad \text{scan}_2\ s\ f\ e\ a = \text{scan}\ s\ f\ e\ a\ .\text{proj}_2$$

3.4 Scan at work

We illustrate the above implementation by applying it to a two-dimensional array of natural numbers, using addition as the binary operation and 0 as the neutral element (Fig. 1). Colours

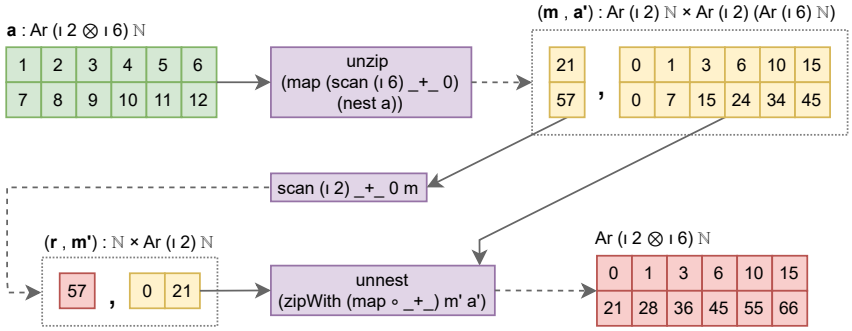


Fig. 1. Application of rank-polymorphic scan to 2-dimensional array of natural numbers.

distinguish arrays from operations. The array a (green, top-left) of shape $(\iota\,2 \otimes \iota\,6)$ is the input. It is fed to a function (purple) that maps `scan` across the rows of a and `unzip`s the array of pairs into a pair of arrays. Solid arrows indicate inputs to functions; striped arrows indicate outputs. The result is a pair (shown in a dotted box, top-right) consisting of partial sums (carries) and scanned rows. We denote the projections of this pair by m and a' respectively. Both m and a' are coloured yellow, signalling temporary data. Scanning the partial sums m produces a pair containing the overall sum r and a scan of the partial sums m'. The overall sum r is coloured red because it forms the first output of the whole scan. Finally, we adjust the rows of a' by adding the corresponding element from m' to each row, *i.e.* adding 0 to the

first row and 21 to the second row. We transform the nested array into an array of product shape using `unnest`. This adjusted array constitutes the second output of the scan and is likewise coloured red.

3.5 *Initial scan*

As `scan` is defined for all models, it can be instantiated for the initial model with the canonical combinators:

```
module IS = Scan I.A (CanonComb I.A) (CanonCombOk I.A)
```

This instance of `scan` is noteworthy because we can prove that, for any array model A and for any semantically-respecting combinators, the results computed by `scan` for that model coincide with those computed by the initial scan. Formally:

```
scan-ok : (s : I.S) → (f : X → X → X) → (e : X) → (a : Ar ⟦ s ⟧s X)
        → scan₁ s f e a ≡  IS.scan₁ s f e ⟦ a ⟧a'
        × scan₂ s f e a ≈ₐ ⟦ IS.scan₂ s f e ⟦ a ⟧a' ⟧a
```

In essence, the theorem states that all scans in all models compute the same values: whenever two models A and B share the same (lifted) shape and the same element values, their respective scans produce identical results. We prove the theorem by induction on the shape s, using the fact that the combinators respect their intended semantics (*Ok*) to show that the translation artefacts introduced by moving between models do not affect the computed values. The theorem would also follow from the parametricity (Reynolds 1983) of `scan` over array models together with the initiality of the initial model. However, Agda does not internalise parametricity, so using this argument would require defining an embedded language, re-expressing the array theory and combinators within it, and proving a general parametricity result à la Bernardy et al. (2010). The direct proof is considerably simpler.

4 Reshaping

One common operation on arrays is an element-preserving change of shape that we call reshape. In principle, reshape can be regarded as an isomorphism between the index spaces of two corresponding arrays. However, in many situations we are interested only in a subset of all possible permutations. In particular, we frequently consider an equivalence relation on shapes such that reshapes within the same equivalence class preserve certain operations, for example `scan`.

Reshape relations make it possible to achieve this by defining a "language" of shape transformations, each of which induces a permutation of array elements. The relation presented below captures those reshapes that preserve array flattening. The reshape relation plays a central role in the stability proof in Section 5.

For any array model A, Reshape is an inductive relation that relates shapes s and p when the products of these shapes (*i.e.* the numbers of elements in the flattened arrays) are identical.

```
data Reshape : S → S → Set where
  eq  : Reshape s s
  _⊕_ : Reshape s p → Reshape q r → Reshape (s ⊗ q) (p ⊗ r)
  _·_ : Reshape q p → Reshape s q → Reshape s p
```

```
split : Reshape (ι (m * n)) (ι m ⊗ ι n)
flat  : Reshape (ι m ⊗ ι n) (ι (m * n))
```

The relation is equipped with an identity constructor eq, composition _•_, gluing of two individual reshapes into a reshape of a product _⊕_, and, most importantly, split and flat which give rise to reversible array flattenings.

With the given definition of Reshape we can show that it is symmetric:

```
rev : Reshape s p → Reshape p s
rev eq       = eq
rev (r ⊕ r₁) = rev r ⊕ rev r₁
rev (r • r₁) = rev r₁ • rev r
rev split    = flat
rev flat     = split
```

Consequently, Reshape is an equivalence relation: it is reflexive by eq, symmetric by rev, and transitive by _•_. Hence shapes together with Reshape form a setoid (in the type-theoretic sense of Section 2.2), allowing us to employ the convenient reasoning syntax demonstrated in the leftn proof below. A number of interesting shape relations can be derived from the properties of the natural numbers. As an illustration, consider the relation between a singleton shape and its product with a singleton ι 1. We use flat to turn _⊗_ into multiplication on natural numbers and exploit the neutrality of multiplication to finalise the definition.

```
leftn : ∀ n → Reshape (ι 1 ⊗ ι n) (ι n)
leftn n = begin
            ι 1 ⊗ ι n ≈⟨ flat ⟩
            ι (1 * n) ≡⟨ cong ι (*-identityˡ _) ⟩
            ι n
          ■ where open ReshapeReasoning
```

Reshape has the following contra-variant action on indices:

```
_⟨_⟩ : P p → Reshape s p → P s
i ⟨ eq     ⟩ = i
i ⟨ r ⊕ r₁ ⟩ = (fst i ⟨ r ⟩) ⊛ (snd i ⟨ r₁ ⟩)
i ⟨ r • r₁ ⟩ = i ⟨ r ⟩ ⟨ r₁ ⟩
i ⟨ split  ⟩ = ϕ $ combine (ψ $ fst i) (ψ $ snd i)
i ⟨ flat   ⟩ = let a , b = remQuot _ (ψ i) in ϕ a ⊛ ϕ b
```

As can be observed, eq has no effect; _⊕_ acts on the corresponding sub-indices; composition _•_ applies two actions in sequence; and split and flat employ an isomorphism between Fin(m * n) and Fin m × Fin n given by combine and remQuot. The function combine maps a pair (i, j) to the row-major offset $i * n + j$, and remQuot recovers i and j from an offset k as the quotient and the remainder of the division of k by n.

Finally, reshape on arrays is defined by means of selection on reshaped indices:

```
reshape : Reshape s p → Ar s X → Ar p X
reshape r a = imap λ ix → a [ ix ⟨ r ⟩ ]
```

We show that reshaping forwards followed by reshaping backwards, and vice versa, yields the identity, via the following lemmas:

$$\langle\rangle\text{-rev} \quad : (r : \mathrm{Reshape}\ s\ p)\ (i : \mathrm{P}\ p) \to i \langle\, r \,\rangle \langle\, \mathrm{rev}\ r \,\rangle \equiv i$$
$$\langle\rangle\text{-rev'} \quad : (r : \mathrm{Reshape}\ s\ p)\ (i : \mathrm{P}\ s) \to i \langle\, \mathrm{rev}\ r \,\rangle \langle\, r \,\rangle \equiv i$$
$$\text{rev-reshape-}{\approx} : (r : \mathrm{Reshape}\ s\ p)\ (a : \mathrm{Ar}\ s\ X) \to \mathrm{reshape}\ (\mathrm{rev}\ r)\ (\mathrm{reshape}\ r\ a) \approx_a a$$
$$\text{reshape-rev-}{\approx} : (r : \mathrm{Reshape}\ s\ p)\ (a : \mathrm{Ar}\ p\ X) \to \mathrm{reshape}\ r\ (\mathrm{reshape}\ (\mathrm{rev}\ r)\ a) \approx_a a$$

The second and the fourth lemmas follow from the first and the third ones, as rev is an involution: $\mathrm{rev}\ (\mathrm{rev}\ r) \equiv r$. Moreover, we prove that reshapes are congruent: if two arrays contain the same elements, then reshaping them with a given r produces arrays that also contain the same elements:

$$\text{reshape-cong} : \{a\ b : \mathrm{Ar}\ s\ X\}\ (r : \mathrm{Reshape}\ s\ p) \to a \approx_a b \to \mathrm{reshape}\ r\ a \approx_a \mathrm{reshape}\ r\ b$$

The proof is immediate: the element of reshape $r\ a$ at index i is the element of a at index $i \langle\, r \,\rangle$, which coincides with the element of b at the same index.

Note that Reshape is a proof-relevant relation, which means that, in general, different expressions of the type Reshape $s\ p$ may exhibit distinct actions on indices and arrays. For the Reshape defined above this situation does not arise, as we prove in Section 4.1. Intuitively, all reshapes preserve the order of elements in the flattened array: they only change how these elements are split into groups, as described by the shape tree. Consequently, no Reshape can describe the transposition of an array, as transposition changes the order of elements in the flattened array.

For example, consider the array $\big[[1,2,3],[4,5,6]\big]$ of shape $(\iota\ 2 \otimes \iota\ 3)$. There exists a Reshape $(\iota\ 2 \otimes \iota\ 3)\ (\iota\ 3 \otimes \iota\ 2)$; however, its action yields the array $\big[[1,2],[3,4],[5,6]\big]$ rather than the desired transposition $\big[[1,4],[2,5],[3,6]\big]$.

If we were to extend Reshape with a `swap` constructor that exchanges the left and right components of a product shape, we could define a transpose operation and, more generally, arbitrary permutations of shape components. Note that this still does not capture arbitrary permutations of array elements, only those induced by permuting the shape. In that scenario we would have two reshapes of the same type that behave differently. Furthermore, `scan` would no longer be stable under reshapes: swapping changes the order of elements in the flattened array, so the prefixes of the reshaped array contain different elements. For instance, scanning the transposed array above and transposing the result back yields $\big[[0,5,12],[1,7,15]\big]$, whereas the scan of the original array is $\big[[0,1,3],[6,10,15]\big]$. This cannot be remedied by requiring additional properties of the binary operation: commutativity would only preserve the overall reduction (the carry), but not the scanned elements. We therefore do not include `swap` in Reshape.

In general, one should envisage many reshape relations—potentially organised hierarchically—that serve distinct purposes. Categorically, shapes and reshapes (identified up to their action) form a groupoid: eq gives identities, $_\cdot_$ gives composition, and rev gives inverses. The proof-irrelevance of the action of our Reshape means that this groupoid is *thin*: there is at most one morphism between any two shapes, so it carries no more information than the equivalence relation itself. Extensions such as `swap` break thinness, and statements about such relations have to refer to particular reshapes rather than to the mere existence of one.

4.1 Initial reshapes

We write IR for the reshapes of the initial model, *i.e.* the definitions of this section instantiated with I.A. As with shapes and indices, reshapes within the initial model can be translated into reshapes within any other model as follows:

```
⟦_⟧r : IR.Reshape s p → Reshape ⟦ s ⟧s ⟦ p ⟧s
```

As with indices, translated reshapes act on lifted shapes.

Furthermore, we can show that lifting reshaped arrays is the same as reshaping lifted arrays with lifted reshapes:

```
reshape-⟦⟧  : (r : IR.Reshape s p) (a : I.Ar s X)
            → ⟦ IR.reshape r a ⟧a ≈ₐ reshape ⟦ r ⟧r ⟦ a ⟧a
reshape-⟦⟧' : (r : IR.Reshape s p) (a : Ar ⟦ s ⟧s X)
            → ⟦ reshape ⟦ r ⟧r a ⟧a' I.≈ₐ IR.reshape r ⟦ a ⟧a'
```

While this is not surprising, it requires a formal proof, which can be found in the accompanying sources (Šinkarovs and Scholz 2026).

Finally, we show that the action of lifted reshapes is proof-irrelevant: any two reshapes between the same shapes act identically on indices, and therefore on arrays:

```
⟨⟩-prop      : (r r' : IR.Reshape s p) (i : P ⟦ p ⟧s)
             → i ⟨ ⟦ r ⟧r ⟩ ≡ i ⟨ ⟦ r' ⟧r ⟩
reshape-prop : (r r' : IR.Reshape s p) (a : Ar ⟦ s ⟧s X)
             → reshape ⟦ r ⟧r a ≈ₐ reshape ⟦ r' ⟧r a
```

We first prove the corresponding property (IR.⟨⟩-prop) in the initial model. Every initial index has a row-major offset, computed by combine-ing the offsets of its sub-indices. This mapping is injective, and a straightforward induction on reshapes shows that every reshape action preserves offsets. Hence, $i \langle r \rangle$ and $i \langle r' \rangle$ have the same offset, and thus they are equal. The lifted version follows by translating indices into the initial model and back.

5 Reshape independence of scan

In this section we demonstrate that scan is stable under reshapes; that is, any reshaping of the input array expressible with Reshape has no effect on the values computed by the scan.

We fix a monoid with binary operation $_\langle+\rangle_$ and neutral element ϵ. We then prove that initial scans are stable under initial reshapes. Two theorems are employed: init-scan$_1$-ok ensures that the carries are equal, whereas init-scan$_2$-ok ensures that the array elements are equal:

```
init-scan₁-ok : (a : Ar s X) (r : Reshape s p)
  → IS.scan₁ p _⟨+⟩_ ϵ (reshape r a) ≡ IS.scan₁ s _⟨+⟩_ ϵ a
init-scan₂-ok : (a : Ar s X) (r : Reshape s p)
  → IS.scan₂ p _⟨+⟩_ ϵ (reshape r a) ≈ₐ reshape r (IS.scan₂ s _⟨+⟩_ ϵ a)
```

Both proofs proceed by induction on the reshape r. For the eq, $_\oplus_$ and $_\cdot_$ constructors the argument follows straightforwardly from the inductive hypothesis. The split and flat constructors, however, require additional reasoning, because we must relate the application of the operation to sub-vectors with the operation applied to the whole vector. The key lemma

states that scanning a vector of length $m * n$ yields the same carry as scanning each of its m consecutive chunks of length n and then scanning the resulting m carries; a similar lemma relates the elements of the scanned vectors. This is precisely the place where associativity of $_\langle+\rangle_$ and neutrality of ϵ are needed: the two computations combine the same elements in the same order, but with a different bracketing. Having proven stability for the initial model, we lift it to all models. The proof concerning carries proceeds as follows. We invoke scan-ok to convert to an initial scan over lifted arrays; then we translate lifted reshapes into initial ones using reshape-⟦⟧' together with the congruence of the initial scan (IS.scan$_1$-cong); the stability of initial scans is applied to eliminate the reshaping, and finally we apply scan-ok in reverse:

```
scan-thm₁ : ∀ s p → (a : Ar ⟦ s ⟧s X) (r : IR.Reshape s p)
          → scan₁ p _⟨+⟩_ ϵ (reshape ⟦ r ⟧r a) ≡ scan₁ s _⟨+⟩_ ϵ a
scan-thm₁ s p a r = begin
  scan₁    p _⟨+⟩_ ϵ (reshape ⟦ r ⟧r a)      ≡⟨ scan-ok p _ _ _ .proj₁ ⟩
  IS.scan₁ p _⟨+⟩_ ϵ ⟦ reshape ⟦ r ⟧r a ⟧a'  ≡⟨ IS.scan₁-cong _ _ _ _ (reshape-⟦⟧' r a) ⟩
  IS.scan₁ p _⟨+⟩_ ϵ (IR.reshape r ⟦ a ⟧a')  ≡⟨ init-scan₁-ok _ r ⟩
  IS.scan₁ s _⟨+⟩_ ϵ ⟦ a ⟧a'                ≡˘⟨ scan-ok s _ _ _ .proj₁ ⟩
  scan₁    s _⟨+⟩_ ϵ a
  ■ where open ≡-Reasoning
```

The proof for arrays follows exactly the same structure as the preceding theorem, and is therefore omitted:

```
scan-thm₂ : ∀ s p → (a : Ar ⟦ s ⟧s X) (r : IR.Reshape s p)
          → scan₂ p _⟨+⟩_ ϵ (reshape ⟦ r ⟧r a) ≈ₐ reshape ⟦ r ⟧r (scan₂ s _⟨+⟩_ ϵ a)
```

As the action of our reshape relation is proof-irrelevant, the choice of a particular reshape between two given shapes does not matter. For any two reshapes r and r' of type Reshape $s\ p$, scanning the array a reshaped by r computes the same results as scanning a reshaped by r'; informally, `scan` (reshape $r\ a$) ≈ `scan` (reshape $r'\ a$). For carries this follows directly from scan-thm$_1$, which holds for every reshape. For the scanned arrays, scan-thm$_2$ shows that the two results are the scan of a reshaped by r and by r' respectively, and these coincide by reshape-prop from Section 4.1. Both statements are proven in the accompanying sources (Šinkarovs and Scholz 2026). Note that reshape relations need not have proof-irrelevant actions in general, so this is not a general result: it relies on the particular Reshape chosen in this paper.

Our rank-polymorphic specification of scan can be viewed as an infinite family of algorithms, one for each array shape. The Reshape equivalence partitions shapes into equivalence classes, and the foregoing proof establishes that scan applied to any two shapes within the same class yields identical results. In the "what/how" metaphor, the *what* remains unchanged whilst the *how* may be systematically varied through the application of reshape.

6 Runtime effects

In this section we demonstrate how the runtime properties of scan can be controlled by reshaping the input array. To make observations we extend our specification by introducing effects that we expect to appear during actual execution. These effects do not interfere with

the computations themselves: once the effects are erased, we recover the original scan, so all the previous results remain applicable.

Effects are introduced via an abstract monad M, together with a function sync that traverses (McBride and Paterson 2008) arrays with effects in M:

```
module Effectful (M : Set → Set) (isMon : Monad M)
                 (sync : ∀ {s X} → I.Ar s (M X) → M (I.Ar s X)) where
```

M is a module parameter, as is the function sync, which is the analogue of `sequenceA` in Haskell for arrays. Consequently, all functions in this module are parametric (Reynolds 1983) in M and cannot exploit its internal structure. The only "interface" to M consists of return, _»=_ and sync. By instantiating M as the identity monad, all effects disappear and we recover the original definition of scan; we prove this formally below. We work in the initial model, and as before we first define an effectful scan on vectors and then on arrays of arbitrary rank. The source of effects is the binary operation (the first argument of scan), which now returns $M\ X$ instead of X, and the algorithm must propagate these effects; consequently the return type of scan is wrapped in M.

When scanning vectors we propagate effects left-to-right, sharing the computation of the i-th element with that of the $(i + 1)$-st:

```
scanv : (X → X → M X) → X → Vec n X → M (X × Vec n X)
scanv {n = zero } f e v = return (e , nil)
scanv {n = suc n} f e v = do e'     ← f e (hd v)
                             s , v' ← scanv f e' (tl v)
                             return (s , cons e v')
```

In the rank-polymorphic scan, one-dimensional arrays are handled by scanv. For arrays of product shapes we apply scan to all sub-arrays (conceptually in parallel), then gather the effects produced by these computations using sync. We bind the partial results to m and a' so that m and a' share the computation. Next we scan the array of carries m (obtaining r and m') and adjust the sub-arrays of a' with the elements of m' (again conceptually in parallel) before accumulating all the effects of this adjustment:

```
scan : (X → X → M X) → X → Ar s X → M (X × Ar s X)
scan {s = ι n}     f e a = scanv f e a
scan {s = s ⊗ p} f e a = do m , a' ← unzip <$> (sync $ map (scan f e) (nest a))
                            r , m' ← scan f e m
                            a''    ← sync $ unnest $ zipWith (map ∘ f) m' a'
                            return (r , a'')
```

Comparing this with the implementation from Section 3.3, we have replaced lets with dos and introduced accumulation of effects via sync in two places. When M is the identity monad and sync is the identity function, the above definition collapses to the earlier one. We make this precise by instantiating Effectful accordingly (we call the resulting module E) and proving that the effectful scan computes the same results as the initial scan from Section 3.5:

```
scan-id : ∀ s (f : X → X → X) (e : X) (a : I.Ar s X)
        → E.scan f e a .proj₁ ≡ IS.scan₁ s f e a
        × E.scan f e a .proj₂ ≈ₐ IS.scan₂ s f e a
```

The proof proceeds by induction on the shape s; the full proof can be found in the accompanying sources (Šinkarovs and Scholz 2026).

6.1 *Left and right nesting*

To illustrate runtime differences when evaluating arrays of differing shapes, we define reshapes that convert arbitrary shape trees into left- and right-nested forms. Conceptually, this technique of right/left traversals applied to scans can be found in Elliott (2017); however, here we achieve the alteration of the algorithm by a single reshape, since we have an explicit handle on the array shape.

First we define a reshape that flattens a shape tree into a singleton shape. We introduce a function that computes the product of the tree's elements, denoted $_\flat$, and the corresponding reshape, called $\flat_r$:

$$
\begin{array}{ll}
_\flat : \mathrm{S} \to \mathbb{N} & \flat_r : \mathrm{Reshape}\ s\ (\iota\ (s\ \flat)) \\
(\iota\ n)\quad \flat = n & \flat_r\ \{\iota\ n\}\quad\ = \mathrm{eq} \\
(s \otimes p)\ \flat = s\ \flat\ *\ p\ \flat & \flat_r\ \{s \otimes p\} = \mathrm{flat} \cdot (\flat_r \oplus \flat_r)
\end{array}
$$

Next we define a helper reshaping that reassociates shapes consisting of three sub-shapes. The definition flattens all three components, applies associativity of multiplication, and then unflattens the shapes:

$$
\begin{array}{l}
\mathrm{assoc}_r : \mathrm{Reshape}\ ((s \otimes p) \otimes q)\ (s \otimes (p \otimes q)) \\
\mathrm{assoc}_r\ \{s\}\ \{p\}\ \{q\} = \\
\quad \mathrm{begin} \\
\qquad (s \otimes p) \otimes q \qquad\qquad \approx\langle\ (\flat_r \oplus \flat_r) \oplus \flat_r\ \rangle \\
\qquad (\iota\ (s\ \flat) \otimes \iota\ (p\ \flat)) \otimes \iota\ (q\ \flat) \quad \approx\langle\ \mathrm{flat} \cdot (\mathrm{flat} \oplus \mathrm{eq})\ \rangle \\
\qquad (\iota\ ((s\ \flat\ *\ p\ \flat)\ *\ q\ \flat)) \qquad \equiv\langle\ \mathrm{cong}\ \iota\ (\mathbb{N}.*\text{-assoc}\ (s\ \flat)\ _\ _)\ \rangle \\
\qquad (\iota\ (s\ \flat\ *\ (p\ \flat\ *\ q\ \flat))) \qquad \approx\langle\ (\mathrm{eq} \oplus \mathrm{split}) \cdot \mathrm{split}\ \rangle \\
\qquad (\iota\ (s\ \flat) \otimes (\iota\ (p\ \flat) \otimes \iota\ (q\ \flat))) \quad \approx\langle\ \mathrm{rev}\ \flat_r \oplus (\mathrm{rev}\ \flat_r \oplus \mathrm{rev}\ \flat_r)\ \rangle \\
\qquad (s \otimes (p \otimes q)) \\
\quad \blacksquare\ \mathrm{where\ open\ ReshapeReasoning}
\end{array}
$$

With these helpers we can define operations that nest shape trees to the right. The left-pointing triangle in the notation indicates the leaf direction. Analogous to flattening, we define the operation on shapes $_\blacktriangleleft$ using the helper $_\triangleleft_$, which right-nests the first argument while leaving the second unchanged:

$$
\begin{array}{ll}
\triangleleft : \mathrm{S} \to \mathrm{S} \to \mathrm{S} & _\blacktriangleleft : \mathrm{S} \to \mathrm{S} \\
(\iota\ n)\quad \triangleleft\ q = \iota\ n \otimes q & (\iota\ n)\ \blacktriangleleft\quad\ = \iota\ n \\
(s \otimes p)\ \triangleleft\ q = s \triangleleft (p \triangleleft q) & (s \otimes p)\ \blacktriangleleft = s \triangleleft (p\ \blacktriangleleft)
\end{array}
$$

We lift these functions to reshapes, obtaining $\otimes\Rightarrow\triangleleft_r$ and $\blacktriangleleft_r$ as follows:

$$
\begin{array}{ll}
\otimes\Rightarrow\triangleleft_r : \mathrm{Reshape}\ (s \otimes p)\ (s \triangleleft p) & \blacktriangleleft_r : \mathrm{Reshape}\ s\ (s\ \blacktriangleleft) \\
\otimes\Rightarrow\triangleleft_r\ \{\iota\ n\}\quad\ = \mathrm{eq} & \blacktriangleleft_r\ \{\iota\ n\}\quad\ = \mathrm{eq} \\
\otimes\Rightarrow\triangleleft_r\ \{s \otimes p\} = \otimes\Rightarrow\triangleleft_r \cdot (\mathrm{eq} \oplus \otimes\Rightarrow\triangleleft_r) \cdot \mathrm{assoc}_r & \blacktriangleleft_r\ \{s \otimes p\ \} = \otimes\Rightarrow\triangleleft_r \cdot (\mathrm{eq} \oplus \blacktriangleleft_r)
\end{array}
$$

For left-nested shape trees the code is symmetric:

```
_▹_ : S → S → S
s ▹ (ι n)     = s ⊗ ι n
s ▹ (p ⊗ q) = (s ▹ p) ▹ q

_► : S → S
(ι n) ►     = ι n
(s ⊗ p) ► = (s ►) ▹ p
```

```
⊗⇒▹r : Reshape (s ⊗ p) (s ▹ p)
⊗⇒▹r {s} {ι n}     = eq
⊗⇒▹r {s} {p ⊗ q} = ⊗⇒▹r · (⊗⇒▹r ⊕ eq)
                    · rev assocr

►r : Reshape s (s ►)
►r {ι n}     = eq
►r {s ⊗ p} = ⊗⇒▹r · (►r ⊕ eq)
```

We leave it as an exercise for the reader to verify that the above code indeed computes the intended nestings; a proof can be found in the paper source (Šinkarovs and Scholz 2026). An example application of both functions would be:

```
sample-shape : S
sample-shape = (ι 2 ⊗ ι 3) ⊗ ((ι 4 ⊗ ι 11) ⊗ (ι 5 ⊗ ι 6) ⊗ ι 7)

right-nested : sample-shape ◄ ≡ ι 2 ⊗ (ι 3 ⊗ (ι 4 ⊗ (ι 11 ⊗ (ι 5 ⊗ (ι 6 ⊗ ι 7)))))
right-nested = refl

left-nested : sample-shape ► ≡ (((((ι 2 ⊗ ι 3) ⊗ ι 4) ⊗ ι 11) ⊗ ι 5) ⊗ ι 6) ⊗ ι 7
left-nested = refl
```

6.2 *Cost model*

Finally we construct a monad that enables reasoning about the complexity and parallelism of scan. The monad M combines a writer (Wadler 1995) and a reader. The writer component counts the number of binary operations performed during the scan, while the reader component records whether the evaluation proceeds sequentially or in parallel:

```
data Par : Set where
  goPar goSeq : Par
```

```
M : Set → Set
M X = Par → ℕ × X

return : X → M X
return x p = 0 , x
```

```
_»=_ : M X → (X → M Y) → M Y
(m »= f) p = let n , x = m p
                 m , y = f x p
             in  m + n , y
```

The Par datatype therefore distinguishes sequential (goSeq) from parallel (goPar) execution. The natural number paired with X in M "carries" the operation count. The return of M performs no binary operations, incurring zero cost in either mode; the bind operator _»=_ executes its argument in the chosen mode, then runs the resulting function in the same mode, summing the operation counts of both invocations. We define sync to model parallel execution on an idealised CPU with a fixed number of threads that run in parallel. Within each thread the code runs sequentially, and both start-up and synchronisation costs are assumed to be zero. Two helper operations, vpar and vseq, compute respectively the maximum and the sum of the first projections in a given vector. Conceptually, this reflects the idea that the time required to execute n items of work in parallel is dictated by the slowest item, whereas the

time for sequential execution is simply the sum of the individual runtimes:

```
vpar : Vec n (ℕ × X) → ℕ × Vec n X
vpar v = fold ℕ._⊔_ 0 (proj₁ ∘ v)
       , proj₂ ∘ v

vseq : Vec n (ℕ × X) → ℕ × Vec n X
vseq v = fold ℕ._+_ 0 (proj₁ ∘ v)
       , proj₂ ∘ v
```

We assume our idealised CPU has eight threads. The scheduling algorithm for a vector of n work items proceeds as follows. When executing items in parallel we split them evenly among the threads and run each sub-vector sequentially; when executing sequentially we simply process each item in order and accumulate the total runtime. For arrays we first flatten them into vectors and then apply the same strategy:

```
vsync : Vec n (M X) → M (Vec n X)
vsync {n} v goPar
  rewrite m≡m%n+[m/n]*n n 8 {{ it }} = let
    r , q = vsplit {n % 8} v
    #r , r' = vseq (λ i → r i goSeq)
    #q , q' = vpar λ j → vseq λ i →
                vnest {m = n / 8} q i j goSeq
  in #r + #q , vjoin r' (vunnest λ i j → q' j i)
vsync v goSeq = vseq (λ i → v i goSeq)

sync : Ar s (M X) → M (Ar s X)
sync a p = let
  n , t = vsync (reshape ♭ᵣ a) p
  in n , reshape (rev ♭ᵣ) t
```

In vsync we employ the theorem m≡m%n+[m/n]*n to rewrite the length n as $n\%8 + (n/8) * 8$. The function vsplit then divides the vector into two parts of lengths $n\%8$ (the remainder) and $(n/8) * 8$. Subsequently vnest reshapes the latter into a nested vector of size $n/8 \times 8$. We run the eight sub-vectors of length $n/8$ in parallel (each sub-vector is processed sequentially). The time taken for the remainder is added to the overall runtime, assuming the slowest thread handles it. Finally we reassemble the results using vjoin and vunnest, which are inverses of vsplit and vnest. With M, its monad structure given by return and _»=_, and sync at hand, we instantiate the module Effectful from the beginning of this section, which gives us the effectful scan for this cost model.

We run scan on an array a of natural numbers with shape (ι 10 ⊗ ι 10 ⊗ ι 10), where every entry equals one. The binary operation is addition of unit cost, denoted _+'_, and the neutral element is zero.

```
a : Ar (ι 10 ⊗ ι 10 ⊗ ι 10) ℕ
a = K 1

_+'_ : (a b : ℕ) → M ℕ
(a +' b) _ = 1 , a + b
```

Helper functions #s and #p compute the cost of scanning sequentially and in parallel, respectively. In the sequential case the cost is the number of binary operations; in the parallel case it is the modelled runtime, *i.e.* the number of operations performed by the slowest thread, summed over all synchronisation points:

```
#s : Ar s ℕ → ℕ
#s a = scan _+'_ 0 a goSeq .proj₁

#p : Ar s ℕ → ℕ
#p a = scan _+'_ 0 a goPar .proj₁
```

We now compare the operation counts for right- and left-nested reshapes of a:

```
_ : #s (reshape ◄ᵣ a) ≡ 3110
_ = refl

_ : #s (reshape ►ᵣ a) ≡ 2210
_ = refl
```

As can be seen, we have a significant difference in operation counts. For an array of shape $s \otimes p$, scan scans all the sub-arrays of shape p, recursively scans the carries of shape s, and then performs one operation per element to adjust the sub-arrays. In the right-nested variant the recursion descends into the sub-arrays, which together contain all the n elements, so every level of nesting costs $O(n)$ operations and, with $O(\log n)$ levels, the total is $O(n \log n)$. In the left-nested variant the recursion descends into the array of carries, which shrinks by a constant factor at every level, so the costs form a geometric series bounded by $O(n)$. When running the same algorithms in parallel we obtain:

```
_ : #p (reshape ◄ᵣ a) ≡ 765        _ : #p (reshape ►ᵣ a) ≡ 341
_ = refl                           _ = refl
```

Parallel execution reduces the modelled runtime of both variants. However, parallelism does not remove the additional work of the right-nested variant, so the left-nested version remains considerably more efficient.

We have illustrated a single runtime model and two reshapes. Nevertheless, the effectful rank-polymorphic code defined above permits further performance engineering by identifying shape structures that best match the execution model of interest as well as adjusting hardware models. In general, a monad with sync may prove insufficient; additional properties could be required. When runtime depends on the actual values stored in arrays, the proposed monadic approach may need to be extended. For simple cases one can define return and _»=_ operations that inspect the values of a particular element type. While useful in practice, such operations break the contract imposed by the monad laws (which our framework does not enforce). A more general solution would involve employing a richer effect system for such modelling.

7 Related work

The array theory presented herein draws inspiration from earlier work on containers (Altenkirch et al. 2015). We restrict the shapes of containers to products of natural numbers, and our reshapes correspond to Cartesian morphisms (Abbott et al. 2003). Containers prove extremely useful when modelling arrays. On the one hand, an explicit notion of shapes and indices affords a straightforward representation of arrays in concrete memory — we merely need to translate indices of a given shape into offsets. On the other hand, because arrays are modelled as functions, reasoning about array expressions becomes simpler; we can frequently exploit properties such as map-fusion through normalisation. Further developments of arrays as containers can be found in Šinkarovs (2020). As this work deals with finite arrays, the theory of combinatorial species (Yorgey 2014)—which are essentially finitary containers—is also relevant. Moreover, since our shapes possess a monoidal structure, the theory of operads (May 1972) can be employed to interpret the proposed arrays.

Due to limited space we presented only a single running example that demonstrates the application of our technique. Nevertheless, a comparable treatment of arrays has proved valuable in the following two scenarios. In Šinkarovs et al. (2023), we consider an algorithm for blocked matrix multiplication. Much as with scan, the tree structure of array shapes enables the encoding of a blocking strategy for the input matrices. We define a high-level algorithm for blocked matrix multiplication, prove its correctness, and use it as a guide to derive an implementation in the array language SaC that performs on par with state-of-the-art libraries

such as OpenBLAS and MKL. In Šinkarovs and Henriksen (2025), we employ a similar array framework to define convolutional neural networks using high-level array combinators. We provide an automatic-differentiation facility in Agda together with a semantics-preserving optimiser and a code generator targeting Futhark for the language of array combinators.

Many ideas that appear in Elliott (2017) resonate with the spirit of this paper. Elliott defines a language of (generic) functors comprising zero, one, and singleton, closed under sums, products and composition. For a given algorithm such as scan, one then defines an instance of the algorithm for every case of that functor language. Consequently, the chosen algorithm can be computed for data structures expressible as generic functors. Containers are isomorphic (Abbott et al. 2005) to strictly positive functors, so all generic operations can be defined on containers as well. The key differences in the present work are threefold: (i) The dependently-typed framework permits reasoning about rank polymorphism — knowledge of array shapes may be deferred until run-time — whereas the decomposition into generic functors advocated by Elliott must occur statically; (ii) Here we abstract over array representation — by selecting a model and combinators the algorithm is guaranteed to compute on the chosen representation — whereas computation via generic functors proceeds by translation, leaving optimisation to the compiler and thereby introducing separate practical challenges; (iii) Most importantly, we furnish a formal proof that isomorphic representations (as given by reshapes) yield identical results, whereas with generic functors one must manually ensure, for example, that scans over `Pair × Pair` and `Pair ∘ Pair` coincide. Finally, our use of monads provides precise numbers that can be taken into account when adapting specifications to a particular hardware model.

There exists a substantial body of array languages that support rank polymorphism to varying extents. Early untyped languages such as APL (Iverson 1962), J (Stokes 2015), and Nial (Smillie 1988) provide run-time rank polymorphism without any type or conformance guarantees. They offer a rich repertoire of rank-polymorphic array combinators and are valuable for rapid prototyping. Within this tradition, Trenchard More initiated the development of array theories (More 1973) that focus on identifying universal equalities in the context of built-in rank-polymorphic combinators. This line of work culminated in Lenore Mullin's Ψ-calculus (Mullin 1988), which is based on a universal abstraction of iteration spaces akin to the imap operator from Section 2.2.

Inspired by these ideas, many functional array languages strive to support rank polymorphism: SaC (Scholz 2003), Qube (Trojahner and Grelck 2011), Accelerate (McDonell et al. 2015), Jeremy Gibbons' Naperian Functors (Gibbons 2016), Remora (Slepak et al. 2014), and ATL (Liu et al. 2022). While languages such as SaC or Accelerate primarily target the generation of efficiently executable code, the others are chiefly concerned with providing static guarantees against out-of-bounds errors. A fully generic setting in which ranks may only be known at run-time necessitates dependent types, as embodied by the array theory presented in this paper or by the type system of Qube. When ranks are restricted to static (compile-time) values, many static guarantees can be obtained with a less expressive type system. For instance, Gibbons (2016) offers a Haskell framework — similar in spirit to our work — for handling statically-ranked multidimensional arrays. Remora (Slepak et al. 2014) proposes a novel type system (without full dependent types) for statically-ranked arrays, furnishing numerous APL-specific features such as shape lifting. ATL (Liu et al. 2022) is a statically-ranked Coq-embedded language that provides safe static indexing. In principle, all

of the aforementioned languages could implement a static variant of the scan specification presented here. SaC, for example, demonstrates (Šinkarovs and Scholz 2022) the expressive power of rank polymorphism. Nonetheless, reproducing our equivalence proof based on reshapes within those systems would likely be challenging.

Futhark (Henriksen 2017) is a functional array language that prioritises parallel performance and type safety. In contrast to the languages discussed above, it does not support rank polymorphism at all, but it does provide a built-in scan primitive. Although convenient for users, this primitive often renders it difficult or impossible to control the run-time behaviour of a particular application.

Hinze (2004) explores the compositional nature of the in-place algorithm introduced by Blelloch (1990). It would be interesting to investigate whether a tidy translation exists between our specification and the one presented in that paper.

Šinkarovs and Cockx (2021) demonstrate that specifications from a framework akin to the one presented here can be translated into high-performance languages. This technique, termed "extraction", leverages the reflection capabilities of the underlying dependently-typed language. With a modest amount of effort, Agda specifications can be compiled into an array language of the user's choice.

Our encoding of arrays bears a strong resemblance to the encoding of type theories using Categories with Families (CwFs) (Castellan et al. 2021). This is unsurprising, since arrays are essentially functions: imap behaves like λ, and _[_] (selection) behaves like function application. Accordingly, the analogy proceeds as follows:

Con : Set	Sh : Set
Sub : Con → Con → Set	Reshape : Sh → Sh → Set
Tm : Con → Ty → Set	Ar : Sh → Set → Set

Array shapes play the role of contexts, and array indices correspond to variables. Reshapes act as substitutions that permit the remapping of variables/indices within an array expression. Arrays correspond to terms, with one difference that is visible in the signatures above: terms inhabit a closed universe of object-level types Ty, whereas array elements may be of any Agda type (Set). The crucial insight for both systems is that substitutions possess an inductive structure, allowing the action of a substitution (*i.e.* reshaping) to be defined inductively as well.

In a sense, the proposed array theory constitutes a rudimentary type theory, wherein the only types that variables may assume in a context (shape) are of the form Fin n for some natural number n. It would be intriguing to develop a closed array theory in which indices and arrays inhabit a single universe. In such a setting, array shapes would themselves be arrays, guaranteeing that any operation on shapes is immediately applicable to arrays. A minimal prerequisite for this construction would be the ability to embed the array theory presented in this paper into such a type theory. Our experiments, however, suggest that attaining this goal might require something akin to very dependent types (Altenkirch et al. 2023).

8 Conclusions and future work

We dressed up rank polymorphism in dependent types, thereby obtaining a haute-couture collection of tools for constructing and formally reasoning about array-based algorithms. This collection comprises: an *array theory* that enables the definition of specifications without

committing to any particular array implementation; the *initial model* of the array theory that supplies recursion over array shapes; *array combinators* defined as interfaces with fixed semantics, ensuring that efficient implementations can be instantiated for all models; and *inductive reshapes* that make it possible to prove theorems concerning the stability of functions under reshapes.

We have demonstrated that our specification of `scan` can be transformed into algorithms exhibiting differing runtime characteristics merely by selecting a shape for the input array. The main theorem guarantees that applying `scan` to any reshaping of the input array that is admissible under Reshape yields identical values. We have shown how monads can be employed to model runtime properties such as sharing and parallelism of array expressions. Coupled with the freedom to choose a runtime representation of arrays and to define efficient combinators for that representation, this approach mitigates guesswork when selecting the optimal shape for a given hardware architecture. An additional benefit of the dependently-typed framework is that all computations are index-safe and all functions are provably terminating.

By no means do we claim that the given array framework is complete for expressing every conceivable numerical computation. Nevertheless, this work offers a perspective on handling collections that can be represented as containers possessing non-trivial shape structure.

The notion of arrays can be generalised by extending shapes with new constructors such as addition, exponentiation, or perhaps even dependent sums or products. By doing so we obtain greater flexibility in grouping elements within arrays, although rank-polymorphic functions and reshapes become correspondingly more intricate. The language of reshapes is not immutable—it may be extended (*e.g.* by permitting permutations) or restricted (*e.g.* by forbidding flattening) according to the problem under consideration. The array combinators presented here remain universal provided that shapes include singleton and product constructors. Moreover, the theory can be readily expanded with additional array combinators without jeopardising the existing development.

The present contribution constitutes a first step toward a wealth of promising future work. Firstly, we aim to generate high-performance code from the proposed specifications. While it is unlikely that Agda alone can produce code competitive with highly optimised low-level libraries, it is straightforward to encapsulate the array definitions within a small DSL and perform code generation, as we have done for convolutional neural networks (Šinkarovs and Henriksen 2025). Another substantial question concerns the translation of scan-like algorithms into in-place array updates as described in Blelloch's original paper (Blelloch 1990). On a theoretical level, it would be interesting to investigate whether incorporating reshape equalities into shape definitions simplifies specifications; however, this would necessitate moving to more expressive type theories that support quotient types.

Conflicts of interest

None.

References

Michael Abbott, Thorsten Altenkirch, and Neil Ghani. 2003. Categories of Containers. In *Foundations of Software Science and Computation Structures*, Andrew D. Gordon (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 23–38.

Michael Abbott, Thorsten Altenkirch, and Neil Ghani. 2005. Containers: Constructing strictly positive types. *Theoretical Computer Science* 342, 1 (2005), 3–27. doi:10.1016/j.tcs.2005.06.002 Applied Semantics: Selected Topics.

Agda Development Team. 2026. *Agda 2.8.0 documentation*. https://agda.readthedocs.io/en/v2.8.0/ Accessed [2026/10/31].

Thorsten Altenkirch, Neil Ghani, Peter Hancock, Conor McBride, and Peter Morris. 2015. Indexed containers. *Journal of Functional Programming* 25 (2015), e5. doi:10.1017/S095679681500009X

Thorsten Altenkirch, Ambrus Kaposi, Artjoms Šinkarovs, and Tamás Végh. 2023. The Münchhausen Method in Type Theory. In *28th International Conference on Types for Proofs and Programs (TYPES 2022) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 269)*, Delia Kesner and Pierre-Marie Pédrot (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 10:1–10:20. doi:10.4230/LIPIcs.TYPES.2022.10

Jean-Philippe Bernardy, Patrik Jansson, and Ross Paterson. 2010. Parametricity and dependent types. In *Proceedings of the 15th ACM SIGPLAN International Conference on Functional Programming* (Baltimore, Maryland, USA) *(ICFP '10)*. Association for Computing Machinery, New York, NY, USA, 345–356. doi:10.1145/1863543.1863592

Guy E. Blelloch. 1990. *Prefix Sums and Their Applications*. Technical Report CMU-CS-90-190. Carnegie Mellon University, USA.

Simon Castellan, Pierre Clairambault, and Peter Dybjer. 2021. Categories with Families: Unityped, Simply Typed, and Dependently Typed. In *Joachim Lambek: The Interplay of Mathematics, Logic, and Linguistics*, Claudia Casadio and Philip J. Scott (Eds.). Springer International Publishing, Cham, 135–180. doi:10.1007/978-3-030-66545-6_5

Conal Elliott. 2017. Generic Functional Parallel Algorithms: Scan and FFT. *Proc. ACM Program. Lang.* 1, ICFP, Article 7 (aug 2017), 25 pages. doi:10.1145/3110251

Jeremy Gibbons. 2016. APLicative Programming with Naperian Functors (Extended Abstract). In *Proceedings of the 1st International Workshop on Type-Driven Development* (Nara, Japan) *(TyDe 2016)*. Association for Computing Machinery, New York, NY, USA, 13–14. doi:10.1145/2976022.2976023

Alexander Grothendieck. 1960. Technique de descente et théorèmes d'existence en géométrie algébriques. II. Le théorème d'existence en théorie formelle des modules. In *Séminaire Bourbaki : années 1958/59 - 1959/60, exposés 169-204*. Number 5 in Astérisque. Société mathématique de France, 369–390. https://www.numdam.org/item/SB_1958-1960__5__369_0/ Exposé no. 195.

Troels Henriksen. 2017. *Design and Implementation of the Futhark Programming Language*. Ph. D. Dissertation. University of Copenhagen, Copenhagen, Denmark.

Ralf Hinze. 2004. An Algebra of Scans. In *Mathematics of Program Construction*, Dexter Kozen (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 186–210.

Kenneth E. Iverson. 1962. *A Programming Language*. John Wiley & Sons, Inc., New York, NY, USA.

Kenneth E. Iverson. 1980. Notation as a Tool of Thought. *Commun. ACM* 23, 8 (Aug. 1980), 444–465. doi:10.1145/358896.358899 1979 ACM Turing Award Lecture.

Amanda Liu, Gilbert Louis Bernstein, Adam Chlipala, and Jonathan Ragan-Kelley. 2022. Verified tensor-program optimization via high-level scheduling rewrites. *Proc. ACM Program. Lang.* 6, POPL, Article 55 (Jan. 2022), 28 pages. doi:10.1145/3498717

J. Peter May. 1972. *The Geometry of Iterated Loop Spaces*. Lecture Notes in Mathematics, Vol. 271. Springer, Berlin, Heidelberg. doi:10.1007/BFb0067491

Conor McBride and James McKinna. 2004. The view from the left. *Journal of Functional Programming* 14, 1 (2004), 69–111. doi:10.1017/S0956796803004829

Conor McBride and Ross Paterson. 2008. Applicative programming with effects. *Journal of Functional Programming* 18, 1 (2008), 1–13. doi:10.1017/S0956796807006326

Trevor L. McDonell, Manuel M T Chakravarty, Vinod Grover, and Ryan R Newton. 2015. Type-safe Runtime Code Generation: Accelerate to LLVM. In *Haskell '15: The 8th ACM SIGPLAN Symposium on Haskell*. ACM, 201–212.

Trenchard More. 1973. Axioms and theorems for a theory of arrays. *IBM J. Res. Dev.* 17, 2 (March 1973), 135–175. doi:10.1147/rd.172.0135

L.M. Restifo Mullin. 1988. *A Mathematics of Arrays*. Ph. D. Dissertation. Syracuse University.

Chris Okasaki. 1998. *Purely functional data structures*. Cambridge University Press, USA.

John C Reynolds. 1983. Types, abstraction and parametric polymorphism. In *Information Processing 83, Proceedings of the IFIP 9th World Computer Congress*, R. E. A. Mason (Ed.). North-Holland, 513–523.

Sven-Bodo Scholz. 2003. Single Assignment C: Efficient Support for High-level Array Operations in a Functional Setting. *J. Funct. Program.* 13, 6 (Nov. 2003), 1005–1059. doi:10.1017/S0956796802004458

Artjoms Šinkarovs and Sven-Bodo Scholz. 2026. Rank polymorphism at work: sources and Agda code. https://rank-polymorphism.github.io/scan-paper/

Justin Slepak, Olin Shivers, and Panagiotis Manolios. 2014. An Array-Oriented Language with Static Rank Polymorphism. In *Programming Languages and Systems*, Zhong Shao (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 27–46.

K. W. Smillie. 1988. Array theory and the Nial programming language. *BIT* 28, 3 (Sept. 1988), 439–449. doi:10.1007/BF01941127

Roger Stokes. 2015. Learning J. An Introduction to the J Programming Language. http://www.jsoftware.com/help/learning/contents.htm. [Accessed 2023/02].

Wouter Swierstra. 2022. A well-known representation of monoids and its application to the function "vector reverse". *Journal of Functional Programming* 32 (2022), e10. doi:10.1017/S0956796822000065

The Agda Community. 2026. *The Agda standard library, version 2.3*. https://github.com/agda/agda-stdlib Accessed [2026/10/31].

Kai Trojahner and Clemens Grelck. 2011. Descriptor-Free Representation of Arrays with Dependent Types. In *Implementation and Application of Functional Languages*, Sven-Bodo Scholz and Olaf Chitil (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 100–117.

Artjoms Šinkarovs. 2020. Multi-dimensional Arrays with Levels. *Electronic Proceedings in Theoretical Computer Science* 317 (May 2020), 57–71. doi:10.4204/eptcs.317.4

Artjoms Šinkarovs and Jesper Cockx. 2021. Choosing is Losing: How to combine the benefits of shallow and deep embeddings through reflection. *CoRR* abs/2105.10819 (2021). arXiv:2105.10819 https://arxiv.org/abs/2105.10819

Artjoms Šinkarovs and Troels Henriksen. 2025. Correctness Meets Performance: From Agda to Futhark. *Proc. ACM Program. Lang.* 9, ICFP, Article 255 (Aug. 2025), 30 pages. doi:10.1145/3747524

Artjoms Šinkarovs, Thomas Koopman, and Sven-Bodo Scholz. 2023. Rank-Polymorphism for Shape-Guided Blocking. In *Proceedings of the 11th ACM SIGPLAN International Workshop on Functional High-Performance and Numerical Computing* (Seattle, WA, USA) *(FHPNC 2023)*. Association for Computing Machinery, New York, NY, USA, 1–14. doi:10.1145/3609024.3609410

Artjoms Šinkarovs and Sven-Bodo Scholz. 2022. Parallel Scan as a Multidimensional Array Problem. In *Proceedings of the 8th ACM SIGPLAN International Workshop on Libraries, Languages and Compilers for Array Programming* (San Diego, CA, USA) *(ARRAY 2022)*. Association for Computing Machinery, New York, NY, USA, 1–11. doi:10.1145/3520306.3534500

Philip Wadler. 1995. Monads for Functional Programming. In *Advanced Functional Programming, First International Spring School on Advanced Functional Programming Techniques-Tutorial Text*. Springer-Verlag, Berlin, Heidelberg, 24–52.

Brent Abraham Yorgey. 2014. *Combinatorial Species and Labelled Structures*. Ph.D. dissertation. University of Pennsylvania, Philadelphia, PA. https://www.cis.upenn.edu/~sweirich/papers/yorgey-thesis.pdf